Лабораторная работа №2
Зусь Александра, гр. 4404

Задание: Напишите программу перемножения больших матриц. Сравните время выполнения последовательной и параллельной программы на 2, 4, 8 потоках (процессорах).

Код программы:

```
#include <iostream>#include <vector>#include <chrono>#include <omp.h>#include <cstdlib>#include <iomanip>#include
<cmath>using namespace std;
using Matrix = vector<vector<double>>;
Matrix create_matrix(int n, double value = 1.0) {
    return Matrix(n, vector<double>(n, value));
}
// Последовательное умножениеvoid multiply_sequential(const Matrix& A, const Matrix& B, Matrix& C, int n) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            double sum = 0.0;
            for (int k = 0; k < n; ++k) {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
}
// Параллельное умножение (по строкам)void multiply_parallel(const Matrix& A, const Matrix& B, Matrix& C, int n)
{#pragma omp parallel for schedule(static)
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            double sum = 0.0;
            for (int k = 0; k < n; ++k) {
                sum += A[i][k] * B[k][j];
            }
            C[i][j] = sum;
        }
    }
}
void print_matrix(const Matrix& M, int n) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            // Если число целое — печатаем без дробной части
            if (M[i][j] == static_cast<int>(M[i][j])) {
                cout << setw(1) << static_cast<int>(M[i][j]) << " ";
            }
            else {
                cout << setw(1) << fixed << setprecision(2) << M[i][j] << " ";
            }
        }
        cout << "\n";
    }
}
int main(int argc, char* argv[]) {
    if (argc != 3) {
        cerr << "Usage: " << argv[0] << " <N> <num_threads>\n";
        return 1;
    }

    int N = atoi(argv[1]);
    int threads = atoi(argv[2]);

    if (N <= 0 || threads <= 0) {
        cerr << "N and num_threads must be positive\n";
        return 1;
    }
```

```
    omp_set_num_threads(threads);

    Matrix A = create_matrix(N, 1.5);
    Matrix B = create_matrix(N, 2.0);
    Matrix C_seq(N, vector<double>(N, 0.0));
    Matrix C_par(N, vector<double>(N, 0.0));

    // Последовательная версия
    auto start = chrono::high_resolution_clock::now();
    multiply_sequential(A, B, C_seq, N);
    auto end = chrono::high_resolution_clock::now();
    double time_seq = chrono::duration<double>(end - start).count();

    // Параллельная версия
    start = chrono::high_resolution_clock::now();
    multiply_parallel(A, B, C_par, N);
    end = chrono::high_resolution_clock::now();
    double time_par = chrono::duration<double>(end - start).count();


    cout << fixed << setprecision(6);
    cout << "Sequential time:    " << time_seq << " s\n";
    cout << "Parallel time:      " << time_par << " s\n";
    cout << "Speedup (seq/par):  " << (time_seq / time_par) << "x\n";

    // Выводим матрицу только если N < 20
    if (N <= 15) {
        cout << "\nResult matrix (parallel):\n";
        print_matrix(C_par, N);
    }

    return 0;
}
```

```
D:\Code\2laba\matrix_mult\x64\Release>matrix_mult.exe 12 2
Sequential time:    0.000001 s
Parallel time:      0.000083 s
Speedup (seq/par):  0.010791x

Result matrix (parallel):
36 36 36 36 36 36 36 36 36 36 36 36
36 36 36 36 36 36 36 36 36 36 36 36
36 36 36 36 36 36 36 36 36 36 36 36
36 36 36 36 36 36 36 36 36 36 36 36
36 36 36 36 36 36 36 36 36 36 36 36
36 36 36 36 36 36 36 36 36 36 36 36
36 36 36 36 36 36 36 36 36 36 36 36
36 36 36 36 36 36 36 36 36 36 36 36
36 36 36 36 36 36 36 36 36 36 36 36
36 36 36 36 36 36 36 36 36 36 36 36
36 36 36 36 36 36 36 36 36 36 36 36
36 36 36 36 36 36 36 36 36 36 36 36
```

Рисунок 1. Демонстрационный запуск: размер матрицы N = 12, число потоков = 2.

Данный скриншот приведён для демонстрации корректности вычисления результирующей матрицы.

В коде установлено ограничение: результирующая матрица выводится только в том случае, если её размер (N) не превышает 15. При больших значениях N (например: 600, 800, 1000 и выше) вывод матрицы отключён.

Это сделано намеренно – чтобы при тестировании на больших размерностях не выводить на экран огромные массивы данных, а лишь подтвердить, что сами вычисления выполняются правильно.

```
D:\Code\2laba\matrix_mult\x64\Release>matrix_mult.exe 1000 2
Sequential time:    1.202602 s
Parallel time:      0.737970 s
Speedup (seq/par):  1.629607x
```

Рисунок 2. Первый запуск: размер матрицы N = 1000, число потоков = 2.

```
D:\Code\2laba\matrix_mult\x64\Release>matrix_mult.exe 1000 4
Sequential time:    1.109889 s
Parallel time:      0.514598 s
Speedup (seq/par):  2.156806x
```

Рисунок 3. Второй запуск: размер матрицы N = 1000, число потоков = 4.

```
D:\Code\2laba\matrix_mult\x64\Release>matrix_mult.exe 1000 8
Sequential time:    1.103718 s
Parallel time:      0.294565 s
Speedup (seq/par):  3.746947x
```

Рисунок 3. Третий запуск: размер матрицы N = 1000, число потоков = 8.

Анализ результатов

В ходе эксперимента проводилось умножение квадратных матриц размера N = 1000 в последовательном и параллельном режимах при разном числе потоков (2, 4 и 8).

Сравнение времени выполнения:

Число потоков	Последовательное время	Параллельное время	Ускорение
2	1.203 с	0.738 с	1.63×
4	1.110 с	0.515 с	2.16×
8	1.104 с	0.295 с	3.75×

Последовательное время во всех запусках остаётся примерно на одном уровне (около 1.1–1.2 с). Параллельное время стабильно уменьшается при увеличении числа потоков.

Объяснение поведения при увеличении числа потоков:

При увеличении количества потоков вычислительная нагрузка распределяется между большим числом ядер процессора, благодаря чему время выполнения параллельной версии заметно сокращается.

Хотя ускорение растёт не строго линейно, оно увеличивается достаточно равномерно. Небольшое отставание от идеальных значений (2×, 4× и 8×) объясняется накладными расходами на создание и синхронизацию потоков, а также ограниченной

пропускной способностью памяти при одновременном доступе нескольких потоков к большим матрицам.

В целом результаты показывают, что использование OpenMP позволяет эффективно ускорить умножение матриц при увеличении числа потоков.